KESHER × GOOGLE AI STUDIO

Master Pro Prompt Collection — One Per AI Element

Version: 1.0 · **Classification:** Operator-Level · **Platform:** Gemini Deep Research + Claude**

Each prompt below is self-contained. Paste it directly into Gemini Deep Research or Claude. Each covers one discrete Google AI Studio / Gemini AI capability and how it maps to Kesher's specific app features.

PROMPT 01 — STRUCTURED OUTPUTS (JSON SCHEMA)

Element: Gemini Structured Output — `response_mime_type: application/json` + JSON Schema enforcement
Platform: Paste into → Gemini Deep Research or Claude

OPERATING CONTRACT

You are the lead AI architect for Kesher — a trust-forward, serious Jewish dating app for the Israeli market. You are conducting a deep technical and product research session on one specific Google AI Studio capability: **Structured JSON Outputs**.

Every claim must be labeled:

- `[FACT — source]` — verified from primary docs or first-party sources
- `[HEURISTIC — reasoning]` — structured inference from evidence
- `[GAP]` — unverified; do not fabricate

Kesher's AI axiom: AI drafts suggestions. Humans approve and send. No AI-generated content reaches users without explicit human action.

WHAT YOU ARE RESEARCHING

The capability: Gemini's ability to return responses constrained to a JSON schema — syntactically guaranteed JSON output via `response_mime_type: "application/json"` and a schema object. Also covers `responseSchema` enforcement, Pydantic/Zod integration via the GenAI SDK, and the distinction between syntactic correctness (guaranteed) vs semantic correctness (not guaranteed).

RESEARCH MISSION — 5 QUESTIONS

Q1 — Technical Deep Dive (Verified)

- What exact API parameters control structured output? (`response_mime_type` , `responseSchema` , `generation_config`)
- Which Gemini models support structured output? Which do not?
- What JSON Schema subset is supported? (anyOf, $ref, recursive schemas — what's allowed, what breaks?)
- What is the difference between "enum mode" and "object mode"?
- What happens when the model produces semantically incorrect but syntactically valid JSON?
- Does streaming work with structured outputs? With what constraints?
- Can structured outputs be combined with tools (Search, URL Context, Code Execution, File Search)? What are the documented incompatibilities?
- What changed in the structured outputs API since August 2024?

Q2 — Kesher Feature Mapping For each Kesher feature below, design the exact structured output schema and explain the architectural decision:

1. **Hebrew bio coach** — 3 draft bios with tone labels and conversation hooks
2. **Values & observance phrasing assistant** — 3 rephrase options + risk phrase detection + clarifier suggestions
3. **"Why this match" transparency cards** — explanation, shared values, suggested first question, forbidden inference list
4. **Toxicity/harassment classifier** — classification, confidence, category, recommended action

5. **Scam/financial solicitation detector** — risk level, detected patterns, recommended action
6. **Pacing coach stop signals** — session state, fatigue level, reflection prompt, recommended action
7. **Private taste profile** — preference weights, explicit signals used, excluded inferences, last updated

For each, produce:

```
Feature: [name]
Schema purpose: [what this output controls]
JSON Schema (full, valid):
{
  // complete schema definition
}
Validation logic required: [what semantic checks must happen after JSON is receiv
Failure mode: [what happens if schema is violated or semantically wrong]
Safety settings required: [which Gemini harm categories to configure]
```

Q3 — Hebrew + RTL Specific Risks

- What encoding issues can occur with Hebrew text inside JSON string values?
- How should RTL strings be handled in JSON (bidi markers, escaping, rendering)?
- Does Gemini's JSON schema enforcement interact differently with non-Latin scripts?
- What are the recommended test cases for Hebrew structured output validation?
- Design a minimal Hebrew structured output test harness: inputs, expected outputs, pass/fail criteria.

Q4 — Production Architecture

- What is the correct server-side architecture for structured outputs in Kesher's stack (Vercel serverless + Supabase + Firebase AI Logic)?
- How does Firebase AI Logic handle structured output — same API parameters?
- What monitoring is required? (schema drift detection, semantic validation errors, latency tracking)
- How should schema versioning work across product releases?
- What is the correct ZDR (Zero Data Retention) configuration for structured output calls containing ISS data (religious observance, behavioral signals)?

Q5 — Prompt Engineering for Structured Output

- What system prompt patterns produce the most reliable structured outputs from Gemini?
- How does temperature setting affect JSON reliability? (recommended range for classification vs generative tasks)
- What "avoid" list should be hardcoded into every Kesher structured output system prompt?
- Design the gold test set: minimum 10 input/expected-output pairs for the Hebrew bio coach structured output.
- What does a "prompt contract" look like for structured outputs in production?

REQUIRED OUTPUT

1. Technical capability summary (300 words max)
2. Supported/unsupported JSON Schema feature matrix
3. Complete schemas for all 7 Kesher features
4. Hebrew/RTL risk inventory with mitigations
5. Production architecture diagram (text-based)
6. Prompt contract template (reusable across features)
7. Gold test set for bio coach (10 pairs minimum)
8. Open gaps requiring live verification

PROMPT 02 — FUNCTION CALLING (TOOL USE)

Element: Gemini Function Calling — custom tool definitions, parallel calling, thought signatures
Platform: Paste into → Gemini Deep Research or Claude

OPERATING CONTRACT

You are the lead AI architect for Kesher — a trust-forward Jewish dating app. You are conducting deep research on one specific Gemini capability: **Function Calling** (also called Tool Use). This is how Gemini calls external functions your code defines, enabling agentic behavior while keeping humans in control.

Kesher's constraint: Gemini function calls are suggestions and triggers for server-side logic. No function call executes an irreversible action (message send, ban, deletion) without a human approval gate.

Every claim: `[FACT — source]` / `[HEURISTIC — reasoning]` / `[GAP]`

WHAT YOU ARE RESEARCHING

The capability: Gemini's ability to call developer-defined functions (tools) — receiving structured function call requests in its response, which your server then executes and returns results from. Includes: tool definition schemas (OpenAPI subset), parallel function calling, sequential calling, thought signatures (Gemini 3 requirement), and the parts array parsing requirement.

RESEARCH MISSION — 5 QUESTIONS

Q1 — Technical Deep Dive

- What is the exact API structure for defining a function/tool in Gemini? (tool declaration format, parameter schemas)
- What OpenAPI subset is supported? What is excluded?
- How does parallel function calling work? What is the correct request/response flow?
- What are thought signatures? Why are they required for Gemini 3 function calling? What error occurs if you omit them?
- How must the `parts` array be parsed? Why "don't assume order" and what does this mean in practice?
- How does function calling interact with structured outputs? Can you get a JSON-constrained response after a tool call round-trip?
- What is the correct multi-turn conversation structure when function calling is in use?

- What changed since August 2024 in function calling behavior?

Q2 — Kesher Feature Mapping Design complete function call tool definitions for each of these Kesher server-side operations:

1. `get_user_profile` — retrieves a user's declared profile data (never behavioral/inferred) for bio coaching context
2. `get_match_signals` — retrieves whitelisted, deterministic match signals for "why this match" explanation (no hidden inference)
3. `log_ai_suggestion_shown` — records that an AI draft was shown to a user (audit trail, opt-out tracking)
4. `flag_for_moderation` — creates a moderation queue entry (human reviewer required; AI cannot resolve)
5. `get_session_metadata` — retrieves session duration and interaction count for pacing coach
6. `get_taste_profile` — retrieves private taste profile (explicit signals only, user has edit/delete rights)

For each, produce:

```
Tool name: [snake_case]
Purpose in Kesher: [what this enables]
Approval gate: [does a human confirm before this executes?]
OpenAPI-compliant definition:
{
  // complete tool definition
}
Return value schema:
{
  // what Gemini will receive back
}
Error handling: [what happens if this tool call fails]
ISS data concern: [does this tool touch ISS data? mitigations?]
```

Q3 — Agentic Moderation Workflow Design a complete function-calling agent workflow for Kesher's moderation queue summarizer:

- System prompt that constrains the agent's authority (suggests only, never acts)
- Tool definitions the moderation agent can call
- Multi-turn conversation flow from report intake to summary output

- Where the human approval gate sits in the flow
- How to prevent the agent from “completing” a moderation action autonomously
- What audit trail must be maintained

Q4 — Thought Signatures in Production

- What exactly is a thought signature in Gemini 3?
- How do you extract and preserve it in a multi-turn function calling conversation?
- What breaks without it?
- Write the correct Node.js/TypeScript code for a function calling round-trip that correctly handles thought signatures
- How should this be tested?

Q5 — Security and Trust Constraints

- What function call injection risks exist? (can a malicious user profile cause Gemini to call unintended tools?)
- How should function inputs be sanitized and validated before execution?
- What functions should NEVER be exposed to Gemini? (e.g., delete_account, send_message, apply_ban)
- Design the function allowlist for each Kesher feature context (bio coach context vs moderation context vs pacing context)
- How does ZDR interact with function calling? Are function arguments retained in abuse monitoring logs?

REQUIRED OUTPUT

1. Technical capability summary (300 words max)
2. All 6 tool definitions (complete, valid, OpenAPI-compliant)
3. Moderation agent workflow (end-to-end)
4. Thought signature handling code (TypeScript)
5. Function allowlist per Kesher context
6. Security threat model for function calling

7. Open gaps requiring live verification

PROMPT 03 — GOOGLE SEARCH GROUNDING

Element: Gemini Search Grounding — built-in Google Search tool, citation handling, 30-day retention
Platform: Paste into → Gemini Deep Research or Claude

OPERATING CONTRACT

You are the lead AI architect for Kesher. You are researching one Gemini capability: **Google Search Grounding**. This allows Gemini to search the web in real time to answer questions with current, cited information — relevant for Kesher's date planning and content-current features.

Critical constraint: Search grounding triggers **mandatory 30-day data retention** that cannot be disabled even with ZDR. This makes it ISS-sensitive for any query containing user location, religious observance, or behavioral data.

Every claim: `[FACT — source]` / `[HEURISTIC — reasoning]` / `[GAP]`

WHAT YOU ARE RESEARCHING

The capability: Enabling Gemini to call Google Search during generation. The model decides when to search, returns responses with inline citations (grounding metadata), and the developer must handle citation display. Includes: cost model (billed per search query after free tier), dynamic retrieval configuration, grounding metadata structure, and the 30-day mandatory retention implication.

RESEARCH MISSION — 5 QUESTIONS

Q1 — Technical Deep Dive

- How is Search grounding enabled in the API? (exact tool configuration)
- What is dynamic retrieval? How does `dynamicRetrievalConfig` and `dynamicThreshold`

work?

- What is the structure of grounding metadata returned? (`groundingChunks` , `webSearchQueries` , `groundingSupports`)
- How must citations be displayed? What are Google's attribution requirements?
- What is the cost model? (free tier, per-query billing)
- Can Search grounding be combined with structured outputs? With URL Context? With File Search?
- What data does Google retain for 30 days and why? What is stored exactly?
- Can ZDR disable the 30-day Search retention? (answer: no — document the exact scope)
- What changed in Search grounding since August 2024?

Q2 — Kesher Feature Mapping For Kesher, Search grounding is relevant only for features that require current, factual, external information. Map each potential use case:

1. **Date venue suggestions** — searching for restaurants, parks, cafes in a given Israeli city
2. **Jewish calendar/holiday awareness** — current Shabbat times, upcoming holidays, local event listings
3. **Community event discovery** — shiurim, cultural events, social gatherings relevant to match interests
4. **Profile coaching context** — searching for recent Israeli cultural references a user mentions

For each, produce:

```
Use case: [name]
Search grounding justified: YES / NO / CONDITIONAL
Justification: [why live search is needed vs static data]
ISS data risk: [does the query contain user location, observance data, or behavio
30-day retention exposure: [what user data would be retained in search logs?]
Privacy mitigation: [how to minimize ISS data in the search query — generalizatio
Query design: [example safe search query that minimizes user data exposure]
Citation display requirement: [how citations must be shown to the user]
Alternative without grounding: [can this be achieved with static data or a curate
Verdict: USE / AVOID / USE WITH STRICT MITIGATIONS
```

Q3 — 30-Day Retention Risk Assessment for Israel

- Under Israeli Amendment 13, location data and religious observance data are ISS. If a search query contains either, 30-day retention in US infrastructure is a cross-border ISS transfer.
- Map every realistic Kesher search query and classify: does it contain ISS data?
- Design a query sanitization layer: how to strip ISS data from search queries before they reach Gemini
- What legal mechanisms (SCCs, explicit consent) must be in place before Search grounding is used for any ISS-adjacent query?
- Is there an Israeli data residency option for Search grounding? [Answer this as FACT if confirmed, GAP if not]

Q4 — Citation and Attribution Architecture

- Design the end-to-end flow for displaying Search-grounded results in Kesher's UI
- What does the grounding metadata structure look like in a real API response?
- What must be displayed to the user? (Google's attribution rules)
- How should citations be rendered in an RTL Hebrew UI?
- What happens if grounding returns no citations? How to handle gracefully?
- Design the error handling and fallback for when Search grounding returns irrelevant or misleading results

Q5 — Alternative Architecture: When NOT to Use Search Grounding

- For each Kesher use case in Q2 that is classified AVOID or conditional: design an alternative that achieves the same result without Search grounding
- Design a curated "Jewish calendar and Israeli venue" database approach that eliminates the 30-day retention problem
- When is a static Supabase table + Gemini structured output preferable to live Search grounding?
- Cost comparison: Search grounding (billed per query) vs static database (zero per-query cost) for Kesher's expected query volume

---

REQUIRED OUTPUT

1. Technical capability summary (300 words max)
2. Complete use case assessment for all 4 Kesher scenarios
3. 30-day retention ISS risk map (query → data type → risk level)
4. Query sanitization layer design
5. Citation/attribution UI architecture for RTL Hebrew
6. Alternative architecture for each Search-avoided use case
7. Open gaps requiring live verification

PROMPT 04 — LIVE API (STREAMING AUDIO / VOICE)

Element: Gemini Live API — bidirectional WebSocket streaming, voice I/O, ephemeral tokens
Platform: Paste into → Gemini Deep Research or Claude

OPERATING CONTRACT

You are the lead AI architect for Kesher. You are researching one Gemini capability: **the Live API** — real-time, low-latency bidirectional audio/text streaming via WebSocket. This is the foundation for Kesher's later-phase voice rehearsal coach feature.

Current status: Hebrew TTS is not available in Gemini Live API. This is a blocking constraint. This research must clarify the full constraints, alternatives, and production readiness criteria before any development decision is made.

Every claim: `[FACT — source]` / `[HEURISTIC — reasoning]` / `[GAP]`

WHAT YOU ARE RESEARCHING

The capability: Gemini Live API enables bidirectional streaming over WebSocket — audio in, audio out, with low latency suitable for conversational voice applications. Includes: session management, ephemeral tokens (required for client-side production security), voice activity

detection (VAD), context window compression, session resumption, and function calling within live sessions.

RESEARCH MISSION — 5 QUESTIONS

Q1 — Technical Deep Dive

- How does the Live API WebSocket connection work? (connection lifecycle, message types, session setup)
- What models are available for Live API? What are their capability differences?
- What is the session duration limit? How does context window compression extend it?
- What is session resumption? How do resumption tokens work?
- How does ephemeral token creation work? What is the token-minting backend pattern?
- What voice selection options exist? What languages support TTS output?
- **Is Hebrew TTS available?** If not, what is the documented language list and is Hebrew expected?
- How does function calling work within a Live API session?
- What is Voice Activity Detection (VAD)? How is it configured?
- What audio formats are supported for input and output?
- Firebase AI Logic Live API: what is its current status (Preview)? What SLA implications does Preview mean?
- What changed in Live API since August 2024?

Q2 — Kesher Voice Rehearsal Coach: Concept Architecture Despite the Hebrew TTS blocker, design the complete conceptual architecture for Kesher's voice rehearsal coach — so the team knows exactly what to build when the blocker is resolved:

```
Feature: Voice Rehearsal / Pre-Date Conversation Coach
Purpose: Allow users to practice conversation topics before a date in a low-stake
User flow:
  [Step-by-step from user entering the feature to ending the session]

System prompt design:
  [What persona, rules, and guardrails the Live API session needs]
```

```
Session architecture:
  [Token minting backend → WebSocket client → session config]

Ephemeral token flow:
  [Backend mints token → client connects → session runs → token expires]

Function calls in session:
  [What functions the coach can call during the conversation]

Session constraints:
  [Max duration, compression strategy, what triggers graceful end]

Privacy design:
  [What audio data is retained? For how long? ZDR applicability?]
  [Is voice data ISS under Amendment 13?]

Hebrew blocker status:
  [Exact documentation of the TTS limitation and what would resolve it]

Fallback design (if Hebrew TTS never arrives):
  [Text-based coaching fallback architecture]
```

Q3 — Hebrew TTS Blocker: Full Investigation

- What languages does Gemini Live API officially support for TTS output?
- Is Hebrew on the roadmap? Is there any official statement?
- What alternative TTS services support Hebrew? (Google Cloud TTS Hebrew support? ElevenLabs? Azure Cognitive Services?)
- Design a hybrid architecture: Gemini Live API for understanding + third-party Hebrew TTS for voice output
- What are the latency, cost, and complexity tradeoffs of the hybrid approach?
- Is text-only Live API (no audio output) viable for a coaching feature? Design the text-based fallback.

Q4 — Production Security Architecture

- Why are long-lived API keys dangerous for client-side Live API connections?
- Design the complete ephemeral token minting backend (Node.js/TypeScript on Vercel serverless)
- What claims/constraints should be encoded in ephemeral tokens?

- Token expiration: what happens to an active session when the token expires?
- How does Firebase AI Logic's App Check interact with Live API security?
- What rate limiting must be applied per user?

Q5 — Go/No-Go Decision Framework Design a checklist for when Kesher should commit to building the voice rehearsal coach:

```
BLOCKER RESOLVED CRITERIA:
☐ Hebrew TTS available in Live API — verified in docs, tested in prototype
☐ Live API reaches GA status (exits Preview) with published SLA
☐ Voice data retention policy clarified and ZDR coverage confirmed
☐ Amendment 13 legal opinion on voice data as ISS obtained
☐ Hebrew audio quality validated by native speakers (minimum 10-person test)
☐ Session resumption stable enough for 10+ minute coaching sessions
☐ Ephemeral token backend deployed and load-tested
☐ Chatfishing perception research completed (is voice coaching seen as manipulati
☐ User research shows demand (>40% of target users interested)

ABORT CRITERIA:
☐ Hebrew TTS not available within [timeline]
☐ Voice data retention creates unacceptable ISS cross-border transfer risk
☐ User research shows voice coaching feels manipulative or inauthentic
```

REQUIRED OUTPUT

1. Technical capability summary (300 words max)
2. Complete voice rehearsal concept architecture (ready to build when blocker resolves)
3. Hebrew TTS blocker documentation (complete, sourced)
4. Hybrid Gemini + Hebrew TTS architecture design
5. Ephemeral token backend code (TypeScript)
6. Go/No-Go decision checklist
7. Open gaps requiring live verification

PROMPT 05 — CONTEXT CACHING

Element: Gemini Context Caching — implicit (auto) and explicit (manual TTL), cost reduction
Platform: Paste into → Gemini Deep Research or Claude

OPERATING CONTRACT

You are the lead AI architect for Kesher. You are researching one Gemini capability: **Context Caching**. This allows frequently-used system prompts and context to be cached, reducing token costs by 75–90% for stable content reused across many requests. Critical for a dating app with thousands of daily AI feature calls.

Every claim: `[FACT — source]` / `[HEURISTIC — reasoning]` / `[GAP]`

WHAT YOU ARE RESEARCHING

The capability: Gemini supports two caching modes — Implicit (automatic, introduced Gemini 2.5+, no configuration required, no storage cost) and Explicit (manual `cachedContent` with TTL control, storage cost applies). Both reduce input token costs for repeated stable context. The cache applies to the prefix of a request — the system prompt and any shared context that doesn't change per user request.

RESEARCH MISSION — 5 QUESTIONS

Q1 — Technical Deep Dive

- What is the difference between implicit and explicit context caching?
- For implicit caching: what triggers it? How do you know if a cache hit occurred? Is there any control over it?
- For explicit caching: what API calls create, use, update, and delete a cached content object?
- What models support each caching mode?
- What is the minimum cacheable token count?
- What content can be cached? (system prompts, documents, conversation history — what is and isn't cacheable?)

- What is the cost model for each? (implicit: no storage cost; explicit: storage cost per token per hour — exact pricing)
- What are TTL options for explicit caching? Minimum/maximum?
- Does context caching interact with ZDR? Does cached content fall under 55-day abuse monitoring?
- What changed in context caching since August 2024?

Q2 — Kesher Cost Model Design the complete caching strategy for each Kesher AI feature, estimating cost impact:

For each feature:

```
Feature: [name]
Cacheable content: [what is stable across all requests for this feature]
Cache type: IMPLICIT / EXPLICIT / BOTH
Estimated token size of cacheable prefix:
TTL recommendation (if explicit):
Estimated daily call volume at scale (1K / 10K / 100K DAU):
Cost without caching (estimated):
Cost with caching (estimated):
Cost reduction %:
Implementation notes:
```

Features to cover:

1. Hebrew bio coach system prompt + few-shot examples
2. Values phrasing assistant system prompt
3. Toxicity classifier system prompt + hate speech taxonomy
4. Scam detector system prompt + known scam pattern reference
5. "Why this match" explanation system prompt
6. Pacing coach system prompt + Jewish calendar context
7. Moderation queue summarizer system prompt + policy reference

Q3 — ZDR and ISS Interaction

- Does cached content (system prompts, few-shot examples) fall under Gemini's 55-day abuse monitoring retention?
- If a system prompt contains any user-identifying information — does that create a

privacy risk?

- **Design rule:** what should NEVER be placed in a cached system prompt for Kesher? (no user PII, no ISS data, no individual behavioral signals)
- What is the correct separation between cached context (stable, non-personal) and per-request context (user-specific, non-cached)?
- Does ZDR coverage extend to cached content? Or does cache storage have different retention rules?

Q4 — Implementation Architecture Design the complete caching implementation for Kesher's server-side API layer:

- Where do cached content objects live? (server-side, per-environment)
- How should cache invalidation work when system prompts are updated (model updates, prompt contract changes)?
- Design the cache management pipeline: create → use → monitor → update → delete
- How do you detect cache hits vs misses in production monitoring?
- What should be logged about cache usage? (without logging user data)
- Supabase + Vercel architecture: where does cache management logic live?

Q5 — Prompt Engineering for Maximum Cache Efficiency

- What prompt structures maximize cache hit rates?
- How should few-shot examples be ordered to maximize prefix caching?
- What is the "cache-friendly" way to structure a Kesher system prompt?
- Write a cache-optimized version of the Hebrew bio coach system prompt
- Write a cache-optimized version of the toxicity classifier system prompt
- What should NOT be in the cached prefix? (per-user data, timestamps, session IDs)

REQUIRED OUTPUT

1. Technical capability summary (300 words max)
2. Cost model table for all 7 Kesher features (at 1K / 10K / 100K DAU)
3. ZDR/ISS interaction rules (what can and cannot be cached)

4. Cache management pipeline design
5. Two cache-optimized system prompts (bio coach + toxicity classifier)
6. Cache monitoring plan
7. Open gaps requiring live verification

PROMPT 06 — FILE SEARCH (RAG / PRIVATE DOCUMENT Q&A)

Element: Gemini File Search — indexed private document store, RAG pattern, tool-combo limits
Platform: Paste into → Gemini Deep Research or Claude

OPERATING CONTRACT

You are the lead AI architect for Kesher. You are researching one Gemini capability: **File Search** (also called the File Search Store API or RAG tool). This allows Gemini to search indexed private documents — relevant for Kesher's moderation policy RAG and internal knowledge base features.

Critical known constraint: File Search **cannot be combined with Google Search or URL Context tools**. File Search is **not supported in Live API sessions**. These are architectural constraints, not preferences.

Every claim: `[FACT — source]` / `[HEURISTIC — reasoning]` / `[GAP]`

WHAT YOU ARE RESEARCHING

The capability: The File Search Store API allows you to ingest, index, and query private documents via Gemini. You upload documents, they are chunked and embedded into a search store, and Gemini can query the store during generation. Separate from the Files API (which is temporary, 48-hour, raw file upload).

RESEARCH MISSION — 5 QUESTIONS

Q1 — Technical Deep Dive

- How does File Search differ from the Files API? (temporary raw upload vs indexed embedding store)
- How are documents ingested into a File Search store? (API calls, formats supported)
- What happens during indexing? (chunking strategy, embedding model used, storage)
- What are the store limits by tier? (document count, store size, query quotas)
- How is a File Search query executed during a Gemini generation request?
- What does the response look like when File Search is used? (citations, chunks returned)
- **Confirmed incompatibility:** File Search + Search grounding = not supported. Document the exact error/limitation.
- **Confirmed incompatibility:** File Search + Live API = not supported. Document scope.
- Does indexed content persist indefinitely until deleted? Or is there a TTL?
- What are the embedding/indexing costs?
- What changed in File Search since August 2024?

Q2 — Kesher Feature Mapping File Search is appropriate for Kesher's internal knowledge bases — not user content. Map the use cases:

1. **Moderation policy RAG** — moderators query Kesher's community guidelines and moderation playbook during queue review
2. **Jewish calendar and observance reference** — Gemini can query a curated database of Jewish holidays, Shabbat laws, and observance-level descriptions to use in bio coaching and pacing features
3. **Israeli dating law reference** — legal compliance documents for internal use (not user-facing)
4. **Prompt contract knowledge base** — internal docs that inform AI behavior across features

For each, produce:

```
Use case: [name]
User-facing: YES / NO
```

```
Document types to ingest: [format, approximate size]
Query pattern: [how Gemini would query this store during generation]
Update frequency: [how often documents change — affects re-indexing strategy]
ISS data concern: [do any ingested documents contain ISS data?]
Tool combination needed: [does this use case need Search grounding too? If yes —
Verdict: USE FILE SEARCH / USE ALTERNATIVE / CONDITIONAL
Alternative if not File Search: [Supabase full-text search? Static context inject
```

Q3 — Moderation Policy RAG: Full Design Design the complete architecture for Kesher's moderation policy RAG:

- What documents go into the store? (community guidelines, moderation decision playbook, escalation criteria)
- How should documents be structured for maximum retrieval accuracy?
- System prompt design for the moderation RAG query
- How does the moderator-facing UI surface retrieved policy citations?
- How is the store updated when policies change?
- Can the moderation RAG store contain any user data? (answer: no — design the isolation)

Q4 — Two-Stage Architecture for Search + Private RAG Given that File Search and Search grounding cannot be combined:

- Design a two-stage architecture that achieves both: (1) live web search for current information, (2) private document RAG for policy/reference
- Where does each stage run? (separate API calls, sequential)
- How are results merged in the final response?
- What are the latency implications?
- Is this architecture justified for any Kesher feature?

Q5 — Alternative: When Supabase Beats File Search

- For what document types and query patterns is Supabase full-text search a better choice than File Search?
- Design the Supabase-based alternative for the Jewish calendar/observance reference use case
- Cost comparison: File Search embedding costs vs Supabase storage costs at Kesher's scale

- Latency comparison: File Search retrieval vs Supabase pg_trgm full-text search
- When should Kesher use static context injection (putting reference text directly in the system prompt) vs File Search?

REQUIRED OUTPUT

1. Technical capability summary (300 words max)
2. Use case assessment for all 4 Kesher scenarios
3. Complete moderation policy RAG architecture
4. Two-stage Search + RAG architecture design
5. Supabase alternative design for Jewish calendar/observance reference
6. Decision matrix: File Search vs Supabase vs Static Injection
7. Open gaps requiring live verification

PROMPT 07 — BUILD MODE (AI STUDIO APP GENERATION)

Element: Google AI Studio Build Mode — full-stack app scaffold generation, Cloud Run deploy, key exposure risks
Platform: Paste into → Gemini Deep Research or Claude

OPERATING CONTRACT

You are the lead AI architect for Kesher. You are researching one Google AI Studio capability: **Build Mode** — the AI-powered full-stack app generator within AI Studio. This is relevant for rapidly prototyping Kesher's AI feature UIs before production code is written.

Critical constraint: Build Mode is explicitly "vibe coding." Generated code must exit AI Studio immediately — to GitHub, Claude Code review, and the Kesher CI pipeline. Build Mode **never generates production-ready code**. API keys in client-side Build Mode apps are a shared-billing abuse risk.

Every claim: `[FACT — source]` / `[HEURISTIC — reasoning]` / `[GAP]`

WHAT YOU ARE RESEARCHING

The capability: Build Mode in AI Studio generates React/TypeScript full-stack apps with a server runtime, Firebase integration, secrets management, and optional Cloud Run deployment — all from a natural language prompt. The generated app uses "AI Chips" to embed Gemini calls and annotation mode to modify the prompt in production.

RESEARCH MISSION — 5 QUESTIONS

Q1 — Technical Deep Dive

- What exactly does Build Mode generate? (client + server components? specific framework?)
- What is an "AI Chip"? How does it work in the generated app?
- What is "annotation mode"? How can prompt behavior be modified without redeploying?
- What is the secrets management model? How do API keys get into the server runtime?
- What are the Cloud Run deployment options? What does one-click deploy produce?
- **Key exposure risk:** when Build Mode deploys to Cloud Run, all users share the developer's Gemini API key. Document the exact risk and what billing/abuse scenarios this creates.
- What does "ZIP export" produce? Is the code clean enough to commit directly to GitHub?
- What CI/CD workflow is needed after Build Mode export?
- What Firebase integrations are available? (Auth, Firestore, Storage)
- What changed in Build Mode since August 2024?

Q2 — Kesher Prototype Pipeline Design the Build Mode → GitHub → Claude Code → Vercel staging pipeline for each Kesher AI feature prototype:

For each feature, define the Build Mode prompt (what to type into Build Mode) and the exit criteria (when does it leave AI Studio):

1. **Hebrew bio coach prototype** — onboarding screen with 3-draft AI output, RTL support,

draft selection UI

2. **"Why this match" card prototype** — match card UI with explanation panel, dummy data initially
3. **Pacing coach UI prototype** — session timer with fatigue signal and reflection prompt overlay
4. **Moderation dashboard prototype** — queue view with AI-generated summary cards, human action buttons
5. **Taste profile editor prototype** — private preference editor with category sliders, delete option

For each, produce:

```
Feature: [name]
Build Mode prompt: [exact prompt to enter into Build Mode]
AI Chips needed: [which Gemini calls the prototype embeds]
Firebase integrations: [Auth? Firestore? Storage?]
RTL/Hebrew requirement: [how to specify RTL layout in the prompt]
Exit criteria: [what must be true before code moves to GitHub]
Claude Code review checklist: [what Claude Code must check in the generated code]
What Build Mode CANNOT do for this feature: [limits to acknowledge]
```

Q3 — Security Hardening After Export Build Mode generated code is prototype-quality. Design the complete hardening checklist that Claude Code must apply after every Build Mode export:

- Client-side secret detection and removal
- Server-side API key migration (Vercel environment variables, not hardcoded)
- Authentication enforcement (where Build Mode may have left auth optional)
- Input validation for Hebrew text inputs (XSS, injection vectors)
- Output validation (AI response validation before rendering)
- RLS enforcement confirmation (Supabase table-level security)
- Dependency audit (Build Mode may install outdated packages)
- Error boundary completeness
- Accessibility (RTL, Hebrew font loading, contrast)

Q4 — RTL and Hebrew in Build Mode

- Does Build Mode support RTL layout natively in generated React components?
- How should RTL requirements be specified in the Build Mode prompt?
- What CSS/Tailwind patterns produce correct RTL behavior in generated code?
- Design the standard RTL-safe component library specification to include in every Build Mode prompt for Kesher
- What Hebrew font loading requirements should be in the Build Mode prompt?

Q5 — Build Mode vs Lovable: When to Use Which Kesher's stack uses both Lovable and AI Studio Build Mode for prototyping. Design a decision matrix:

```
Decision criterion | Use Lovable | Use AI Studio Build Mode
--------------------|-------------|-------------------------
Feature needs Supabase auth baked in | [answer] | [answer]
Feature is purely AI feature UI demo | [answer] | [answer]
Feature needs to share with stakeholders quickly | [answer] | [answer]
Feature needs production-shaped code to review | [answer] | [answer]
Feature uses Gemini AI Chips directly | [answer] | [answer]
Feature needs Firebase integration | [answer] | [answer]
Team wants to prompt-iterate the UI rapidly | [answer] | [answer]
```

REQUIRED OUTPUT

1. Technical capability summary (300 words max)
2. Build Mode prompt + exit criteria for all 5 Kesher prototypes
3. Security hardening checklist (Claude Code review template)
4. RTL/Hebrew Build Mode specification template
5. Lovable vs Build Mode decision matrix
6. Prototype-to-production pipeline diagram (text-based)
7. Open gaps requiring live verification

PROMPT 08 — CODE EXECUTION (PYTHON

SANDBOX)

Element: Gemini Code Execution — sandboxed Python runtime, data analysis, simulation
Platform: Paste into → Gemini Deep Research or Claude

OPERATING CONTRACT

You are the lead AI architect for Kesher. You are researching one Gemini capability: **Code Execution** — Gemini's ability to write and run Python code in a sandboxed environment during generation, enabling "compute-backed reasoning" for data analysis and simulations.

Kesher relevance: Code Execution is not user-facing. It is an internal capability for evaluating matching algorithms, simulating taste profile outcomes, validating scoring rubrics, and running moderation metric analyses — all without touching production user data.

Every claim: `[FACT — source]` / `[HEURISTIC — reasoning]` / `[GAP]`

WHAT YOU ARE RESEARCHING

The capability: Gemini can write Python code, execute it in a secure sandbox, observe the output, and iterate — returning the result as part of its response. The sandbox is Python-only, has a 30-second execution limit, supports NumPy, Pandas, and Matplotlib, and has no file I/O beyond what's passed in the prompt.

RESEARCH MISSION — 5 QUESTIONS

Q1 — Technical Deep Dive

- How is Code Execution enabled in the API? (exact tool configuration)
- What Python libraries are available in the sandbox?
- What is the execution time limit? Memory limit?
- Can code execution produce files (CSVs, images, charts)?
- How does the model iterate — can it run code, see the output, and rewrite code in the same turn?

- Can Code Execution be combined with other tools? (Search grounding, structured outputs, function calling)
- What are the billing implications? (is there an additional charge for code execution?)
- What are the security guarantees of the sandbox? Can it access the internet? The filesystem?
- What changed in Code Execution since August 2024?

Q2 — Kesher Internal Use Cases Design complete Code Execution workflows for each of these Kesher internal analysis tasks:

1. **Matching algorithm simulation** — given synthetic user profiles (no real data), simulate the Daily Picks ranking algorithm across 1,000 synthetic users and evaluate distribution fairness
2. **Taste profile scoring simulation** — evaluate how taste profile weights affect match set diversity; detect over-filtering scenarios
3. **Pacing coach calibration** — analyze session duration distribution data (aggregated, non-personal) to calibrate burnout thresholds
4. **Toxicity classifier threshold analysis** — given a labeled test set (100 examples), evaluate precision/recall at different confidence thresholds
5. **Cost modeling** — calculate Gemini API costs across features at 1K / 10K / 100K DAU using token count estimates

For each, produce:

```
Use case: [name]
Inputs to the code execution sandbox: [what synthetic/aggregate data is passed in
Task for Gemini to compute: [exact instruction]
Expected output: [chart? JSON? table? recommendation?]
Real user data used: NEVER — document how synthetic data is generated
ISS data risk: [none if all data is synthetic — confirm]
Iteration pattern: [does this need multi-turn code iteration?]
Sample prompt (the instruction to Gemini including the data):
  [complete prompt with synthetic example data inline]
```

Q3 — Evaluation Harness Design Design Kesher's AI feature evaluation harness using Code Execution:

- How to structure a prompt that passes 50+ test cases to Gemini for evaluation
- How to extract pass/fail results from Code Execution output

- How to run regression testing (compare new prompt version vs previous version)
- How to export evaluation results from AI Studio Logs/Datasets
- Design the evaluation dataset format for the Hebrew bio coach (inputs, expected output structure, pass/fail criteria)

Q4 — Safe Data Handling

- What data can safely be passed into a Code Execution sandbox? (synthetic data, aggregate statistics, test vectors)
- What must NEVER be passed in? (real user profiles, PII, ISS data, actual message content)
- Design the synthetic data generation pipeline: how Kesher creates realistic fake user data for algorithm testing
- Is data passed to Code Execution subject to Gemini's 55-day abuse monitoring retention?
- Does ZDR apply to Code Execution inputs?

Q5 — Production vs AI Studio: When Does Code Execution Exit AI Studio?

- For which Kesher analyses does Code Execution in AI Studio stay as an internal tool?
- For which analyses should Code Execution logic be extracted into production code (server-side Python/Node.js) once validated?
- Design the "validate in AI Studio → extract to production" pipeline
- What does the extracted production code look like for the matching algorithm simulation?
- When should Kesher use Code Execution vs running the same computation deterministically in Supabase/Postgres?

REQUIRED OUTPUT

1. Technical capability summary (300 words max)
2. Complete workflow design for all 5 Kesher use cases
3. Evaluation harness design (Hebrew bio coach test set)
4. Synthetic data generation specification

5. AI Studio → Production extraction pipeline
6. Data handling rules (what can/cannot enter the sandbox)
7. Open gaps requiring live verification

PROMPT 09 — SAFETY FILTERS & CONTENT CLASSIFICATION

Element: Gemini Safety Settings — configurable harm categories, threshold control, Hebrew content risks
Platform: Paste into → Gemini Deep Research or Claude

OPERATING CONTRACT

You are the lead AI architect for Kesher. You are researching one Gemini capability: **Safety Filters and Content Classification** — Gemini's built-in harm detection and configurable safety thresholds, plus Gemini's classification capabilities for user-generated content moderation.

Critical context: Gemini 2.5+ models have safety filters **OFF by default**. Kesher must explicitly set restrictive thresholds. Hebrew content safety filter accuracy is undocumented. Religious identity content in Hebrew is a high-sensitivity area.

Every claim: `[FACT — source]` / `[HEURISTIC — reasoning]` / `[GAP]`

WHAT YOU ARE RESEARCHING

The capability: Gemini returns safety ratings with every response across four harm categories: Harassment, Hate Speech, Sexually Explicit, and Dangerous Content. Thresholds are configurable (BLOCK_NONE through BLOCK_LOW_AND_ABOVE). Child safety filters cannot be disabled. Gemini can also be used proactively as a classifier for user-generated content (UGC) in dating apps — classifying inbound messages for toxicity, harassment, and scam signals.

RESEARCH MISSION — 5 QUESTIONS

Q1 — Technical Deep Dive

- What are the four configurable harm categories? What does each cover?
- What threshold levels are available? What does each level block?
- **What is the default for Gemini 2.5+ models?** (answer: OFF — document this precisely)
- What safety categories cannot be disabled? (child safety — document scope)
- How are safety ratings returned in the API response? (structure, fields)
- What is `finishReason: SAFETY` ? How should this be handled in production?
- Can safety settings be configured per-request? Or only at the client/session level?
- Are safety settings configurable for both generation output and input classification?
- What changed in safety filter defaults since August 2024? (major change: default shifted)

Q2 — Kesher Safety Configuration Design the complete safety configuration for each Kesher AI feature:

For each feature, define:

```
Feature: [name]
Direction: OUTPUT (what Gemini generates) / INPUT (classifying user content) / BO
Harassment threshold: [BLOCK_LOW_AND_ABOVE / BLOCK_MEDIUM_AND_ABOVE / BLOCK_ONLY_
Hate Speech threshold: [same options]
Sexually Explicit threshold: [same options]
Dangerous Content threshold: [same options]
Rationale: [why each threshold was chosen]
Religious identity risk: [Hebrew dating content with religious terms — what miscl
finishReason: SAFETY handling: [what happens in the UI if a response is blocked?]
Fallback: [what does the user see if a bio coach draft is safety-blocked?]
```

Features to configure:

1. Hebrew bio coach (output)
2. Values/observance phrasing (output — religious content is high sensitivity)
3. Toxicity classifier (input — classifying user messages)
4. Scam detector (input — classifying user messages)

5. “Why this match” explanation (output)
6. Pacing coach reflection prompts (output)
7. Conversation opener drafts (output)

Q3 — Hebrew Content Safety: Unknown Territory

- What evidence exists that Gemini’s safety filters work correctly in Hebrew?
- What types of Hebrew content are most likely to produce false positives? (religious terminology, Mizrahi slang, Hebrew profanity patterns)
- What types of Hebrew content are most likely to produce false negatives? (harassment in colloquial Hebrew, coded antisemitic language, community-specific insults)
- Design a Hebrew content safety test suite:
  - Minimum 50 labeled test cases
  - Categories: clearly safe / borderline religious content / clear harassment / edge case slang
  - Test methodology
  - Pass/fail criteria for Kesher’s minimum acceptable accuracy
- What is Kesher’s plan if Hebrew safety filter accuracy is below threshold?

Q4 — Using Gemini as a UGC Classifier Beyond Gemini’s own safety filters on its outputs, design Gemini’s use as a classifier for Kesher’s user-generated content:

- Design the toxicity classification system prompt + input format + output JSON schema
- Design the scam/financial solicitation detection system prompt + input format + output JSON schema
- What confidence thresholds trigger automatic hiding vs human review queue vs no action?
- How should Gemini’s UGC classification interact with Kesher’s human moderation workflow?
- What are the false positive and false negative costs for Kesher specifically?
  - False positive (safe message blocked): damages user experience, reduces trust
  - False negative (harmful message passes): damages user safety, damages trust more
- How should Kesher handle classification uncertainty? (confidence in 40–60% range)

Q5 — Production Monitoring and Compliance

- How should safety filter trigger rates be monitored in production?
- What alert thresholds indicate a problem? (high safety block rate = prompt or safety config issue)
- How should safety decisions be logged for compliance? (without retaining the blocked content)
- How does the 55-day abuse monitoring retention interact with safety classification logs?
- What reporting obligations does Kesher have if safety filters detect illegal content? (CSAM — mandatory reporting; other categories — document obligations)
- How should the Hebrew moderation test suite be run as a regression test before every system prompt change?

REQUIRED OUTPUT

1. Technical capability summary (300 words max)
2. Complete safety configuration for all 7 Kesher features
3. Hebrew content safety test suite (50 cases, labeled, with methodology)
4. UGC toxicity classifier prompt contract (system prompt + schema)
5. UGC scam detector prompt contract (system prompt + schema)
6. Confidence threshold → action decision tree
7. Production monitoring and compliance plan
8. Open gaps requiring live verification

PROMPT 10 — FIREBASE AI LOGIC (APP CHECK + SERVER-SIDE PROXY)

Element: Firebase AI Logic — server-side API key protection, App Check, per-user rate limits, mobile/web SDK
Platform: Paste into → Gemini Deep Research or Claude

OPERATING CONTRACT

You are the lead AI architect for Kesher. You are researching one capability: **Firebase AI Logic** — the recommended production gateway for calling Gemini from client applications (web, mobile). This is the layer that keeps Gemini API keys off the client, enforces App Check, and applies per-user rate limits.

Kesher architecture rule: No Gemini API key on the client side. Ever. Firebase AI Logic is the required gateway for any Kesher feature that calls Gemini from a user-facing client.

Every claim: `[FACT — source]` / `[HEURISTIC — reasoning]` / `[GAP]`

WHAT YOU ARE RESEARCHING

The capability: Firebase AI Logic (rebranded May 2025) is Google's SDK for calling Gemini models directly from mobile/web apps in a secure way. It integrates with Firebase App Check (prevents unauthorized callers), Firebase Auth (per-user rate limiting), and Firebase infrastructure (Storage, Firestore, Remote Config). It supports standard Gemini API features plus Live API (Preview).

RESEARCH MISSION — 5 QUESTIONS

Q1 — Technical Deep Dive

- What exactly does Firebase AI Logic do architecturally? (where does the API key live? what does the SDK call?)
- What is App Check? How does it work? What attestation providers are supported (DeviceCheck, Play Integrity, reCAPTCHA)?
- What is the per-user rate limit? (100 RPM default — document and confirm)
- How does Firebase Auth integrate for per-user quotas?
- What SDKs are available? (Swift, Kotlin, JavaScript, Dart, Unity)
- What Gemini features are supported? (structured outputs, function calling, Live API — what works and what doesn't?)
- Live API in Firebase AI Logic: what is the current Preview status and what does "no SLA" mean operationally?

- How does Firebase AI Logic interact with ZDR?
- What changed in Firebase AI Logic since August 2024? (rebranding, capability expansion)

Q2 — Kesher Production Architecture Design the complete Firebase AI Logic production architecture for Kesher's web app (Next.js on Vercel):

```
Architecture diagram (text-based):
[User Browser] → [Next.js Client (no API key)] → [Firebase AI Logic SDK]
                                                        ↓
                                                [App Check attestation]
                                                        ↓
                                                [Firebase Auth (user identity)]
                                                        ↓
                                                [Gemini API (server-side key)]
                                                        ↓
                                                [Response → Client]
```

For each Kesher AI feature, specify:

```
Feature: [name]
Client-side or server-side call: [which architecture pattern]
App Check required: YES / NO (always YES for user-facing)
Auth required: YES (always — Kesher requires auth for all AI features)
Rate limit exposure: [what happens when 100 RPM is hit for a user?]
Firebase integrations used: [Firestore? Storage? Remote Config?]
Live API used: YES / NO
```

Q3 — App Check Implementation Design the complete App Check setup for Kesher:

- Which attestation provider for web? (reCAPTCHA Enterprise — document configuration)
- Which attestation provider for iOS? (DeviceCheck — document configuration)
- Which attestation provider for Android? (Play Integrity — document configuration)
- What happens when App Check fails? (user experience, error handling)
- How do you test App Check in development? (debug token flow)
- What does App Check protect against? (API key abuse, unauthorized callers, bot traffic)
- Design the App Check monitoring: how to detect and respond to App Check failures in production

Q4 — Rate Limiting and Abuse Prevention

- 100 RPM per user is the documented default. Is this sufficient for Kesher's features?
  - Bio coach: called once during onboarding, 1–3 times total per user
  - Toxicity classifier: called on every inbound message — could be 100+ per hour for active users
  - Pacing coach: called once per session
  - Evaluate each feature's expected RPM and whether it fits within the limit
- What happens when a user hits the rate limit? Design the error experience
- How can Supabase server-side rate limiting complement Firebase AI Logic's per-user limits?
- Design the abuse detection layer: patterns that indicate API abuse beyond normal usage
- How should rate limit events be logged for security monitoring?

Q5 — Comparison: Firebase AI Logic vs Direct Vercel Serverless → Gemini API Kesher could also call Gemini from Vercel serverless functions directly (not via Firebase AI Logic). Design a decision matrix:

```
Factor | Firebase AI Logic | Vercel Serverless → Gemini
--------|-------------------|---------------------------
API key location | [answer] | [answer]
App Check support | [answer] | [answer]
Per-user rate limiting | [answer] | [answer]
Latency | [answer] | [answer]
Firebase ecosystem dependency | [answer] | [answer]
Cost | [answer] | [answer]
ZDR compatibility | [answer] | [answer]
Best for Kesher when: | [answer] | [answer]
```

Recommendation: for each Kesher AI feature, specify which architecture to use.

REQUIRED OUTPUT

1. Technical capability summary (300 words max)
2. Complete production architecture diagram
3. App Check implementation guide (web + iOS + Android)
4. Rate limit analysis per feature (RPM requirements vs 100 RPM limit)

5. Firebase AI Logic vs Vercel direct decision matrix
6. Abuse prevention and monitoring plan
7. Open gaps requiring live verification

PROMPT 11 — MAPS GROUNDING

Element: Gemini Maps Grounding — 250M+ places, place cards, 30-day mandatory retention
Platform: Paste into → Gemini Deep Research or Claude

OPERATING CONTRACT

You are the lead AI architect for Kesher. You are researching one Gemini capability: **Maps Grounding** — allowing Gemini to search Google Maps (250M+ places) during generation to return location-aware responses.

Critical privacy constraint: Maps grounding triggers **mandatory 30-day data retention** that cannot be disabled even with ZDR. Location data is ISS under Israeli Amendment 13. This creates a cross-border ISS data transfer risk for any query containing user location data.

Every claim: `[FACT — source]` / `[HEURISTIC — reasoning]` / `[GAP]`

WHAT YOU ARE RESEARCHING

The capability: Gemini can call Google Maps during generation to find places, restaurants, parks, venues, and points of interest. Returns structured place data including place IDs, coordinates, and optionally an interactive map widget token. Shared billing model with Search grounding. Attribution requirements (Google Maps branding) must be displayed.

RESEARCH MISSION — 5 QUESTIONS

Q1 — Technical Deep Dive

- How is Maps grounding enabled in the API? (exact tool configuration)
- What data does Maps grounding return? (place IDs, names, coordinates, ratings, hours, photos?)
- What is the interactive map widget token? How does it work in a client UI?
- Can user location (lat/lng) be passed as context? How?
- What are the attribution requirements? (must display Google Maps branding?)
- What is the cost model? (shares quota with Search grounding — document free tier and billing rate)
- **30-day mandatory retention:** what exactly is retained? (the full prompt including user location? just query metadata?)
- Can ZDR disable this 30-day retention? (answer: no — document precisely)
- Can Maps grounding be combined with Search grounding? With structured outputs? With File Search?
- What changed in Maps grounding since August 2024? (new in Oct 2025 — document what's changed since launch)

Q2 — Kesher Date Planning Feature: Full Architecture Design the complete date planning feature for Kesher using Maps grounding — with all privacy mitigations applied:

```
Feature: AI-Powered Date Planning
User scenario: Two users have matched and want help planning their first date in

Privacy-first query design:
  - User provides: city/neighborhood (NOT precise coordinates)
  - Mutual interests shared between the match (from declared profile data only)
  - Preferred date type (outdoor / dining / cultural)
  - NOT passed to Maps: precise home address, precise current location, observanc

Maps grounding query:
  [Example query designed to minimize ISS data exposure]

Response design:
  - Place suggestions (max 3 options to avoid overwhelm)
  - Place card format (name, vibe description, why it fits their interests)
  - Google Maps attribution display
  - User selects option (AI suggests, user decides)

30-day retention disclosure:
  - What the user is told before triggering Maps grounding
```

```
  - Consent moment design
  - How to make consent feel natural, not scary

Alternative without Maps grounding:
  - Curated Supabase table of pre-approved Israeli date venues
  - No 30-day retention
  - Less current, but privacy-preserving
  - When to use which
```

Q3 — ISS Data Minimization for Maps Queries Design the complete query sanitization layer for Maps grounding in Kesher:

- Input mapping: what user data Kesher has → what can safely go into a Maps query
- Generalization rules: neighborhood instead of address, city instead of neighborhood for sensitive users
- Data that must NEVER be in a Maps query: precise coordinates, home neighborhood if ISS-adjacent, observance level, relationship status
- Design the consent flow for Maps-grounded queries
- Design the data flow audit: how to verify that ISS data is not leaking into Maps queries
- What Israeli legal mechanism covers the 30-day Maps retention? (legal opinion required — document the gap)

Q4 — UI Architecture: Maps Attribution + RTL

- What does Google's Maps attribution requirement mean for a Hebrew RTL interface?
- Design the place card component for Kesher's UI (Hebrew-first, RTL, with Google Maps attribution)
- How does the interactive map widget token work in a Next.js React component?
- What happens if Maps grounding returns no results for a specific Israeli location?
- Design the fallback: user-directed venue search without Gemini (link to Google Maps directly)
- Accessibility requirements for map content in Hebrew

Q5 — Cost and Scalability

- Cost model: Maps grounding at Kesher's expected usage (estimated daily date planning queries at 1K / 10K / 100K DAU)
- At what DAU does the curated Supabase venue database become cheaper than Maps

grounding?

- Design the hybrid approach: curated database for popular Israeli cities (Tel Aviv, Jerusalem, Haifa) + Maps grounding for long-tail locations
- Rate limit considerations: shared quota with Search grounding — how to budget across features

---

REQUIRED OUTPUT

1. Technical capability summary (300 words max)
2. Complete date planning feature architecture
3. ISS data minimization rules (query sanitization layer)
4. Consent flow design
5. RTL/Hebrew place card UI component specification
6. Cost model at three DAU levels
7. Curated venue database alternative design
8. Open gaps requiring legal and technical verification

PROMPT 12 — THINKING CONTROLS (EXTENDED REASONING)

Element: Gemini Thinking Controls — thinking budgets, thought signatures, Gemini 2.5 Pro thinking behavior
Platform: Paste into → Gemini Deep Research or Claude

OPERATING CONTRACT

You are the lead AI architect for Kesher. You are researching one Gemini capability: **Thinking Controls** — the ability to configure how much internal reasoning ("thinking") Gemini performs before generating a response. This affects output quality, latency, and cost.

Kesher relevance: Most Kesher features are low-to-medium complexity (bio coaching, classification) where thinking can be minimized for speed and cost. A few features (matching explanation generation, complex moderation analysis) may benefit from extended thinking.

Every claim: `[FACT — source]` / `[HEURISTIC — reasoning]` / `[GAP]`

WHAT YOU ARE RESEARCHING

The capability: Gemini 2.5 models support thinking budgets — controlling how many tokens the model "thinks" before generating. Gemini 3.1 Pro cannot disable thinking. Thinking tokens are billed differently from output tokens. Thought signatures must be preserved when thinking is used with function calling.

RESEARCH MISSION — 5 QUESTIONS

Q1 — Technical Deep Dive

- What models support thinking controls? (Gemini 2.5 Flash, 2.5 Pro, 3 series — document each)
- What is a "thinking budget"? How is it set in the API?
- Can thinking be disabled completely? On which models?
- **Gemini 3.1 Pro cannot disable thinking** — document the implication
- How are thinking tokens billed? (separate from input/output tokens?)
- What is a thought signature? Why must it be preserved in function calling contexts?
- What is the relationship between thinking budget and output quality? (diminishing returns at high budgets?)
- What latency does thinking add? (rough order of magnitude at different budget levels)
- What changed in thinking controls since August 2024?

Q2 — Kesher Feature Thinking Configuration Design the thinking configuration for each Kesher AI feature:

```
Feature: [name]
Thinking required: YES / NO / MINIMAL
```

```
Thinking budget recommendation: [NONE / LOW (128 tokens) / MEDIUM (1024) / HIGH (
Reasoning:
  - Task complexity: [simple classification / multi-step reasoning / structured g
  - Latency tolerance: [real-time interactive / batch / background]
  - Quality vs speed tradeoff: [what degrades without thinking?]
  - Cost impact: [thinking token cost vs quality gain]
Recommended model: [Gemini 2.5 Flash / 2.5 Pro / 3.1 Flash / etc.]
```

Features to configure:

1. Hebrew bio coach — structured rewriting (low complexity, speed matters)
2. Values phrasing assistant — sensitive content generation (medium)
3. Toxicity classifier — binary classification (very low complexity)
4. Scam detector — pattern detection (low complexity)
5. "Why this match" explanation — multi-factor reasoning (medium)
6. Pacing coach reflection prompts — contextual generation (low)
7. Moderation queue summarizer — synthesis of multiple reports (medium-high)
8. Taste profile generator — multi-factor inference from signals (medium)

Q3 — Thought Signatures in Production

- What exactly is a thought signature and where does it appear in the API response?
- In what scenarios is preserving the thought signature mandatory?
- Write complete TypeScript code for a function calling flow that correctly handles thought signatures
- What error occurs if thought signatures are missing in a multi-turn function calling conversation?
- Design a test to verify thought signature handling in Kesher's API layer

Q4 — Cost vs Quality Analysis

- Design a methodology for determining the optimal thinking budget for each Kesher feature
- What is the expected cost difference between zero thinking and medium thinking for 1M daily classification calls?
- For the Hebrew bio coach: what quality degradation occurs at zero thinking vs 1K

thinking tokens?

- Design the A/B test framework: how to measure quality difference between thinking configurations at Kesher's scale
- What is the "thinking sweet spot" — where additional thinking tokens stop improving quality?

Q5 — Batch Processing with Thinking

- For Kesher's internal batch operations (daily match generation, taste profile updates, moderation batch analysis): how does thinking interact with batch API?
- What thinking configuration is appropriate for batch vs real-time requests?
- Can thinking be disabled for simple batch classification tasks to reduce cost?
- Design the batch processing architecture for daily taste profile refresh across all active users

---

REQUIRED OUTPUT

1. Technical capability summary (300 words max)
2. Thinking configuration table for all 8 Kesher features
3. Thought signature handling code (TypeScript)
4. Cost vs quality analysis methodology
5. Batch processing thinking design
6. A/B test framework for thinking optimization
7. Open gaps requiring live verification

Kesher × Google AI Studio · Master Pro Prompt Collection v1.0 12 prompts · 12 AI elements · One per capability · Trust before growth.